

AcquirerX — End-to-End Testing Playbook

A step-by-step manual testing guide. Anyone on the team should be able to read this top-to-bottom, follow it on a fresh laptop, and validate every feature without needing to ask questions. **No prior knowledge of the codebase or domain is assumed.**

Table of Contents

- [Section A — What is AcquirerX \(in plain English\)](#)
 - [Section B — Glossary of terms](#)
 - [Section C — Roles in this system](#)
 - [Section D — How to set up your machine for testing](#)
 - [Section E — How to start the system](#)
 - [Section F — Reference card \(URLs, ports, roles, sample data\)](#)
 - [Section G — Create the 6 test users \(do this first\)](#)
 - [Section H — How to read a test case in this doc](#)
 - [Module 1 — Identity & Access Management \(IAM\)](#)
 - [Module 2 — Merchant Onboarding, Pricing & Settlement Profile](#)
 - [Module 3 — Terminal / POS Provisioning](#)
 - [Module 4+5 — POS Switch + Fee Engine \(Transactions\)](#)
 - [Module 6 — Settlement, Payouts & Adjustments](#)
 - [Module 7 — Disputes \(Retrieval / Chargeback / Representment / Arbitration\)](#)
 - [Module 8 — Reconciliation & Exceptions](#)
 - [Module 9 — Risk & Fraud](#)
 - [Module 10 — Reports & Dashboards](#)
 - [Module 11 — Notifications](#)
 - [Section I — End-to-end happy-path walkthrough](#)
 - [Section J — Negative & security tests \(RBAC\)](#)
 - [Section K — How to use Postman with this system](#)
 - [Section L — Troubleshooting](#)
 - [Section M — Bug-report template](#)
-

Section A — What is AcquirerX (in plain English)

AcquirerX is a system that a **bank or payment processor** would use to manage all the moving parts of card payments at shops. Think of it as the office software behind every Visa/Mastercard swipe in a retail store. It handles:

- **Onboarding shops** ("merchants") and their physical locations ("stores")
- **Provisioning POS machines** (the card-swipe terminal at the counter, called a "terminal" or "TID")
- **Routing and approving** every card swipe (sometimes called the "switch")
- **Calculating fees** the merchant must pay (interchange fee, scheme fee, acquirer markup)
- **Settling** money to the merchant's bank account once a day
- **Disputes** when a cardholder claims "I never bought this" (chargebacks)
- **Reconciliation** — matching the system's records against the bank's records
- **Risk** — blocking fraudulent transactions
- **Reporting** and notifications

You're testing the operations console — the screens used by bank staff, not the cardholder-facing app.

Section B — Glossary of terms

Read this once. The whole rest of the doc uses these terms.

Term	Meaning	Example
Merchant	A business that accepts cards	"Testing Corporation Pvt Ltd"
Store	A physical location of a merchant	"Downtown Branch" of Testing Corp
Terminal	The card-swipe device at the counter	Verifone V200c
TID	Terminal ID — unique number printed on the device, 8 digits	12345678
MID	Merchant ID — internal number for the merchant	4
MCC	Merchant Category Code — 4-digit code that says what the shop sells	5411 = Grocery, 5812 = Restaurant
PAN	Primary Account Number — the 16-digit number on the card	4532012345670366
Masked PAN	PAN with the middle hidden so it's safe to store	453201*****0366
BIN	First 6 digits of PAN, identifies the card-issuing bank	453201
EMV	Chip card standard ("insert the card")	—
CTLS	Contactless / tap-to-pay	—
Magstripe	Old swipe-the-back-of-the-card method	—
Auth / Authorization	Asking the cardholder's bank "is this card good for ₹X?" before charging	"Auth code 045123"
Batch	A group of approved transactions on one terminal, closed at end of day	—
Network	Card scheme: Visa (V), Mastercard (M), UPI (U), or LocalSim for testing	—
Interchange	Fee the merchant's bank pays the cardholder's bank	typically 1–2%
Scheme fee	Fee Visa/Mastercard charge for using their network	0.05–0.15%
Acquirer markup	Profit margin the bank running AcquirerX adds on top	0.2–0.5%
MDR	Merchant Discount Rate — the total % the merchant pays. MDR = interchange + scheme + markup	2.0%
Settlement	Once-daily process: bundle all approved txns, deduct fees, send the net to the merchant's bank	—

Payout	The actual transfer of money to the merchant's bank account	—
Adjustment	Manual debit/credit on a merchant's settlement (e.g. for a chargeback)	—
Chargeback	Cardholder disputes a charge; the merchant must defend it or refund	—
Dispute stages	1. Retrieval → 2. Chargeback → 3. Representment → 4. Arbitration	—
Reconciliation	Match what AcquirerX thinks happened vs. what the bank's file says happened	—
Exception	A reconciliation mismatch that needs human resolution	MissingTxn / Duplicate / AmountMismatch
JWT	The "ticket" you get after login. Every other API call needs it.	A long string starting with eyJhbGc...

Section C — Roles in this system

Six roles. Each role sees a different sidebar.

Role	What they do
ADMIN	Sees everything. Can create users, change anyone's role, configure fee rules.
MERCHANT_OPS	Onboards merchants and stores; views transactions; pulls merchant reports.
POS_OPS	Provisions terminals and parameter profiles; opens/closes batches; monitors terminal health.
RISK	Reviews risk events and rules; manages PAN/Terminal/Merchant blacklist; views transactions.
DISPUTES	Handles chargebacks/disputes — moves cases through stages, links documents, logs actions.
RECON	Runs settlements, posts adjustments, loads reconciliation files, resolves exceptions.

Sidebar visibility matrix

If a row is  , that page does not appear in the sidebar **and** typing the URL directly redirects to a "Forbidden" page.

Page	ADMIN	MERCHANT_OPS	POS_OPS	RISK	DISPUTES	RECON
Dashboard						
Merchants						
Stores						

Terminals	✓	✗	✓	✗	✗	✗
Transactions	✓	✓	✗	✓	✗	✗
Risk	✓	✗	✗	✓	✗	✗
Disputes	✓	✗	✗	✗	✓	✗
Settlements	✓	✗	✗	✗	✗	✓
Reconciliation	✓	✗	✗	✗	✗	✓
Reports	✓	✓	✗	✗	✗	✓
Notifications	✓	✓	✓	✓	✓	✓
User Management (admin)	✓	✗	✗	✗	✗	✗
Fee Rules (admin)	✓	✗	✗	✗	✗	✗

Section D — How to set up your machine for testing

You need:

1. **Java 17+** (`java -version` should show 17 or higher)
2. **Maven** (`mvn -v`)
3. **Node.js 18+** and **npm** (`node -v` , `npm -v`)
4. **MySQL 8.x** running on `localhost:3306` . Default root password expected: see `application.properties` of any service — default is `kai13114` unless you've changed it.
5. A modern browser (Chrome / Edge / Firefox)
6. **Postman** (only needed for Section K — API tests)

One-time database setup

Open MySQL Workbench (or `mysql` CLI) and run:

```
CREATE DATABASE IF NOT EXISTS acquirerx_auth;
CREATE DATABASE IF NOT EXISTS acquirerx_merchant;
CREATE DATABASE IF NOT EXISTS acquirerx_terminal;
CREATE DATABASE IF NOT EXISTS acquirerx_transaction;
CREATE DATABASE IF NOT EXISTS acquirerx_settlement;
CREATE DATABASE IF NOT EXISTS acquirerx_risk;
CREATE DATABASE IF NOT EXISTS acquirerx_ops;
```

These are empty — the first time each microservice boots, it creates its own tables automatically (Hibernate's `ddl-auto=update`).

Section E — How to start the system

The order matters. Wait for each service to print `Started X in N seconds` before starting the next.

1. Start Eureka (the service registry)

```
cd eureka-server
mvn spring-boot:run
```

Wait until you see `Started EurekaApplication`. Verify by opening <http://localhost:8761> in your browser — you should see Eureka's dashboard.

2. Start the seven microservices (each in its own terminal)

```
cd auth-service      && mvn spring-boot:run    # port 9099
cd merchant-service  && mvn spring-boot:run    # port 9091
cd terminal-service  && mvn spring-boot:run    # port 9092
cd transaction-service && mvn spring-boot:run  # port 9093
cd risk-service      && mvn spring-boot:run    # port 9094
cd settlement-service && mvn spring-boot:run    # port 9095
cd ops-service       && mvn spring-boot:run    # port 9096
```

You can start them in parallel. Wait ~30 seconds, then refresh <http://localhost:8761> — all 7 services should appear under "Application" with status `UP`.

3. Start the API Gateway (after all 7 services show UP)

```
cd api-gateway
mvn spring-boot:run
```

The gateway runs on **port 9090** and is the single entry point for all API calls from the frontend.

4. Start the frontend

```
cd frontend
npm install      # only first time, or when dependencies change
npm run dev
```

Frontend runs on <http://localhost:5173>. Open that URL — you should see a login page.

 **Tip:** keep all the terminals open. If a service crashes, restart only that one. You don't need to restart everything.

Section F — Reference card

Pin this section. You'll come back to it constantly.

URLs & ports

What	URL
Frontend (the UI you test)	http://localhost:5173
API Gateway (Postman uses this)	http://localhost:9090
Eureka dashboard	http://localhost:8761

MySQL	localhost:3306
-------	----------------

Sample data formats — copy/paste these directly

Field	✔ Valid examples	✗ Invalid examples
Username	admin1, risk_user_1, mops42	kail kail (spaces), a (too short), admin@1 (special char)
Password	Admin@123, Test@1234	12345 (under 6 chars)
Email	admin@example.com	not-an-email, a@b
Phone	+91 9876543210, 9876543210, (011) 4567-8900	123 (too short), abc
MCC (4 digits)	5411 (Grocery), 5812 (Restaurant), 5311 (Dept Store)	541, 54111, ABCD
TID (8 digits exact)	12345678, 99887766	123 (too short), abc12345 (letters), 123456789 (too long)
Masked PAN (6 digits + 3-9 of *X + 4 digits)	453201*****0366, 400000XXXXXX1234, 123456***7890, 111111XXXXXXXXXX2222	234932433243*X (mask at end), 1234*****0000 (5-digit prefix), 4532XXX0366 (no middle digits)
Currency (3 uppercase)	INR, USD, EUR	inr, Rupee, INRR
Network (single letter)	v (Visa), m (Mastercard), u (UPI), LocalSim	VISA, RUPAY, Visa
Region	NA, EU, APAC, LATAM	IN, INDIA, Asia
Bank account ref (free text)	HDFC0001234, ICIC-CA-998877	(empty)
Date (form input)	The date picker handles format. Pick from calendar.	Don't type freehand.

Default test data the playbook will create

You'll create these once and reuse them:

Thing	Value
Test merchant	Testing Corporation Pvt Ltd (DBA Testing Corp, MCC 5411)
Test store	Downtown Branch (Bengaluru, KA, 560001)
Test terminal	TID 12345678, model Verifone V200c, capability EMV
Test PAN for transactions	453201*****0366
Default sale amount	1500.00 INR

Section G — Create the 6 test users

Do this **once**. Then you can switch between roles by logging out and logging in as a different username.

Steps to create each user

1. Open <http://localhost:5173> in your browser.
2. You're on the Login page. Click **Register** at the bottom (or go to <http://localhost:5173/register>).
3. Fill the form **exactly** as shown in the table below:

Username	Password	Confirm Password	Full Name	Phone	Email	Role
admin1	Admin@123	Admin@123	Admin User	+91 9000000001	admin1@test.com	Admin
mops1	Mops@1234	Mops@1234	Merchant Ops	+91 9000000002	mops1@test.com	Merchant Ops
pops1	Pops@1234	Pops@1234	POS Ops	+91 9000000003	pops1@test.com	POS Ops
risk1	Risk@1234	Risk@1234	Risk Analyst	+91 9000000004	risk1@test.com	Risk
disp1	Disp@1234	Disp@1234	Disputes Analyst	+91 9000000005	disp1@test.com	Disputes
recon1	Recon@1234	Recon@1234	Recon Analyst	+91 9000000006	recon1@test.com	Recon

4. Click **Register**. You'll land on the login page with a brief flash. Log in with the credentials you just created to verify they work, then log out (click the avatar at top-right → **Logout**).
5. Repeat for the other 5 users.

 **Important:** Phone is required by the form. If you leave it blank the form will reject. Email must be unique across all users — don't reuse the same email twice.

After this, you can simply log out and log in as any role for the rest of the testing.

Section H — How to read a test case in this doc

Every test case has the same shape:

```
### TC-XXX-NN: Short title `[ROLES who can do this]`  
1. Step 1 – exactly which menu, button, field  
2. Step 2 – what to type, what to click  
-  What you should see (success criteria)  
-  What it might wrongly do (failure / edge case)  
-  Optional DB check
```

Symbol	Meaning
--------	---------

[ADMIN]	Only ADMIN can do this. Logged in as another role you'd see "Forbidden".
[ADMIN, RECON]	Either ADMIN or RECON can.
[any role]	Every role can.
✓	Expected (good) behaviour. The test passes if you see this.
✗	Negative or edge case. The test passes if the system blocks/rejects/warns correctly.
🔍	Optional verification — usually a SQL query or DevTools check. Skip if you don't have MySQL Workbench open.
i	Background info.
⚠	Common pitfall. Read it.

Module 1 — Identity & Access Management (IAM)

What this module does: registration, login, role-based access, profile, audit trail.

TC-IAM-01: Self-register a new user [any role]

1. Open <http://localhost:5173/register>.
 2. Fill all fields with valid values (see Section G for examples). Pick any role from the dropdown.
 3. Click **Register**.
- ✓ Page redirects to `/login`. No red toast appears.
 - ✗ If a field has invalid format, an inline red error appears under that field. Form stays open.
 - 🔍 In MySQL: `SELECT user_id, username, name, phone, role, status FROM acquirerrx_auth.app_user WHERE username='<your-username>';` — should show one row with `status=ACTIVE` and your name + phone saved.

TC-IAM-02: Login + JWT contains userId [any role]

1. Go to <http://localhost:5173/login>.
 2. Enter `admin1` / `Admin@123`. Click **Login**.
- ✓ Page redirects to `/dashboard`. Top-right corner shows your username and an `ADMIN` badge.
 - 🔍 Press F12 → Application tab → Local Storage → `http://localhost:5173`. Find key `ax_token`. Copy its value.
 - 🔍 Paste the token at <https://jwt.io>. The decoded payload must contain `sub`, `role`, and `userId`. If `userId` is missing, the dispute action panel won't be able to log who took an action.

TC-IAM-03: Login with wrong password [any role]

1. On login page, enter `admin1` / `wrongpass`.
- ✓ Red toast: "Wrong username or password." Page stays on login. Local Storage `ax_token` is NOT set.

TC-IAM-04: Login on inactive account [any role]

Prereq: TC-IAM-09 has been run (admin deactivated `mops1`).

1. Try to log in as `mops1`.

-  Toast: "Account is inactive."

TC-IAM-05: View my profile [any role]

1. Logged in as any user, click the **avatar (round circle with initials)** at the top-right.
 2. Click **Profile** in the dropdown.
-  The Profile page opens at `/profile` . There are exactly **two tabs**: **Profile** and **Security**. (No Preferences. No Login History. Those were removed.)
 -  Profile tab shows your Full Name, Email, Phone, and Username (greyed out, can't edit).

TC-IAM-06: Update my profile [any role]

1. On the Profile tab, change the **Phone** to `+91 9123456789` . Save.
-  Green message "Profile updated successfully" appears for ~3 seconds.
 -  Refresh the page (F5). The new phone persists.
 -  Same SQL as TC-IAM-01 — the `phone` column shows the new value.

TC-IAM-07: Change my password [any role]

1. Click the **Security** tab on the Profile page.
 2. Current password: `Admin@123` . New password: `NewPass@5678` . Confirm: `NewPass@5678` . Submit.
-  Success message. Log out (click avatar → Logout).
3. Log in with the **new** password.
-  Login works.
4. (Cleanup) Change it back to `Admin@123` so the rest of the playbook still uses the documented password.

TC-IAM-08: User Management list [ADMIN only]

1. Logged in as `admin1` . Sidebar → **User Management** (under "Administration" group at the bottom).
-  A table appears with all users: Username, Email, Name, Role, Status, Created At.
2. Now log out, log in as `mops1` , and try to navigate to <http://localhost:5173/admin/users> directly.
-  The page redirects to a Forbidden screen (sidebar doesn't show this link for non-admins, and direct URL navigation is blocked).

TC-IAM-09: Deactivate / reactivate user [ADMIN]

1. As `admin1` → Sidebar → User Management.
 2. Find the row for `mops1` . Click **Deactivate** (red button) → confirm in the popup.
-  Status badge for that row flips to `INACTIVE` . Toast: "mops1 deactivated".
 -  Now try logging in as `mops1` (in another browser or after logging out) — gives "Account is inactive" (TC-IAM-04 confirmed).
3. Click **Reactivate** on the same row.
-  Status flips back to `ACTIVE` . Login as `mops1` works again.

TC-IAM-10: Change role [ADMIN]

1. As `admin1` → User Management → find `mops1` → click **Change Role**.
 2. In the modal, pick `RISK` from the dropdown → Save.
-  Row's role badge changes to `RISK` . Toast confirms.
3. Log out, log in as `mops1` . Sidebar should now show the **RISK** sidebar (Risk + Transactions + Notifications + Dashboard) — not the Merchant_Ops sidebar.

4. (Cleanup) Change it back to MERCHANT_OPS.

TC-IAM-11: Self-deactivate / self-role-change blocked [ADMIN]

1. As `admin1`, click **Deactivate** on the row for `admin1` (your own row).
- ☒ The button click triggers an error: "You cannot deactivate your own account." (Same for changing your own role.)

TC-IAM-12: Audit logs [ADMIN]

1. As `admin1` → User Management → click on any user (e.g. `mops1`) to open the user detail page.
2. Click the **Audit** tab.
- ☒ The table shows historical events: `REGISTER`, `LOGIN_SUCCESS`, `LOGIN_FAILED`, `USER_DEACTIVATED`, `USER_REACTIVATED`, `ROLE_CHANGED` etc., each with a timestamp and the actor (the admin who performed it).

Module 2 — Merchant Onboarding, Pricing & Settlement Profile

What this module does: create a merchant business, attach its KYC documents, set the fee structure, set the settlement (payout) preferences, and create their physical store locations.

Key business rule (very important to understand)

A new merchant always starts with status = `Pending`. They become `Active` automatically the moment the first KYC document is submitted. So:

```
Create merchant → status: Pending (yellow badge)
Submit any KYC → status: Active (green badge) ← happens automatically
```

This is enforced by the backend; the UI banner on the merchant form mentions it.

TC-MER-01: Create new merchant [ADMIN, MERCHANT_OPS]

1. Log in as `admin1` (or `mops1`).
 2. Sidebar → **Merchants**. Top-right corner click + **New Merchant**.
 3. Fill the form:
 - **Legal Name (required)**: `Testing Corporation Pvt Ltd`
 - **DBA**: `Testing Corp`
 - **MCC**: `5411`
 - **Contact Info (required)**: `ops@testingcorp.com`
 - **Risk Level**: pick `LOW` from the dropdown
 4. Click **Create Merchant**.
- ☒ Page navigates to the merchant detail page (URL like `/merchants/4`).
 - ☒ Big yellow **Pending** badge next to the merchant name (top right).
 - ☒ The status-actions bar shows two buttons: **Activate** and **Suspend** (Activate is the green one).
 -  SQL: `SELECT merchant_id, legal_name, status FROM acquirerrx_merchant.merchant ORDER BY merchant_id DESC LIMIT 1; — status='PENDING'`.

TC-MER-02: MCC validation [ADMIN, MERCHANT_OPS]

1. New Merchant form → MCC = `541` (only 3 digits) → Save.
- ☒ Red error under the MCC field: "MCC must be a 4-digit code (e.g. 5411)". Form does not submit.

2. Try 54111 (5 digits) — same error.
3. Try ABCD — same error.

TC-MER-03: Submit KYC document — auto-activates merchant [ADMIN, MERCHANT_OPS]

1. Open the merchant from TC-MER-01. Status is Pending .
 2. Click the **KYC** tab (one of: KYC | Pricing | Settlement | Stores).
 3. Click + **Submit KYC**.
 4. Fill:
 - **Document Type**: pick PAN_CARD from the dropdown
 - **Document Reference**: AAAC2727Q
 - **Notes** (optional): Verified company PAN
 5. Click **Submit**.
- ☒ KYC row appears in the table with status PENDING (the KYC document itself is pending verification).
 - ☒ ☒ **The merchant status badge at the top of the page flips from Pending to Active (yellow → green)**. This is the key business rule firing.
 -  SQL: SELECT status FROM acquirers_merchant.merchant WHERE merchant_id=<id>; → now ACTIVE .

TC-MER-04: Verify a KYC [ADMIN, MERCHANT_OPS]

1. On the KYC tab, on the row from TC-MER-03 click **Verify**.
- ☒ KYC row's status changes to VERIFIED and Verified Date shows today's date.

TC-MER-05: Reject a KYC [ADMIN, MERCHANT_OPS]

1. Submit a second KYC: Document Type GST_CERT , Ref 27AAAC2727Q1ZT .
 2. On that row click **Reject**.
 3. In the prompt, type a reason: Document image is too blurry to read . Submit.
- ☒ Row status REJECTED . The reason is stored in the notes column.

TC-MER-06: Duplicate KYC blocked [ADMIN, MERCHANT_OPS]

1. Try to submit another PAN_CARD document for the same merchant.
- ☒ Inline error: "PAN_CARD already submitted for merchant: Testing Corporation Pvt Ltd." Form does not save.

TC-MER-07: Add pricing model [ADMIN, MERCHANT_OPS]

1. Merchant detail → **Pricing** tab.
 2. Read the blue info banner — pricing models are immutable historical records. Once deactivated they can't be reactivated.
 3. Click + **Add Pricing Model**.
 4. Fill:
 - **Model Type**: pick MDR from the dropdown
 - **MDR %**: 2.0
 - **Per-Txn Fee**: 2.50
 - **Scheme Fee Pass-Through**: No
 - **Effective From**: today's date (use the date picker)
 - **Effective To**: leave blank
 5. Click **Create**.
- ☒ Pricing row appears with green Active badge.

TC-MER-08: Pricing validation — out of range [ADMIN, MERCHANT_OPS]

1. Try MDR % = 120 .
 - ☒ Error: "MDR cannot exceed 100%".
2. Try MDR % = -1 .
 - ☒ Error: "MDR cannot be negative".

TC-MER-09: Deactivate pricing & re-pricing flow [ADMIN, MERCHANT_OPS]

i *By design, you cannot reactivate a pricing model. It's an immutable historical record (the rate charged on each past transaction must stay auditable). To change the pricing, deactivate the current ACTIVE row and create a new one.*

1. Click the pause-style **Deactivate** icon on the active pricing row.
 - ☒ Status flips to Inactive . The deactivate button disappears (no Reactivate exists, deliberately).
2. Click + **Add Pricing Model** again — fill new values (e.g. MDR 2.25% , Per-txn 2.00 , Effective today).
 - ☒ New row created and Active . Old row stays Inactive forever.
3. Try to add a second Active pricing while one is already active.
 - ☒ Server rejects: "Active pricing model already exists for merchant: ... Deactivate existing model first."

TC-MER-10: Add settlement profile [ADMIN, MERCHANT_OPS]

1. Merchant detail → **Settlement** tab. Read the same kind of "historical record" info banner.
2. Click + **Add Profile**.
3. Fill:
 - **Settlement Cycle**: pick T_PLUS_1 (means money lands in the merchant's bank one day after the txn)
 - **Bank Account Ref (required)**: HDFC0001234-00998877
 - **Reserve %**: 5
4. Save.
 - ☒ Active settlement profile shows T+1, 5% reserve.

TC-MER-11: Update settlement profile [ADMIN, MERCHANT_OPS]

1. On the active settlement profile, click **Edit**. Change cycle to WEEKLY , reserve to 10 . Save.
 - ☒ Saved. Row shows the new values.

TC-MER-11b: Replace settlement profile (deactivate + add new) [ADMIN, MERCHANT_OPS]

1. Click **Deactivate** on the active profile.
 - ☒ Status Inactive . No Reactivate button (intentional).
2. Click + **Add Profile** with new terms (cycle T_PLUS_2 , bank ref ICIC0000999 , reserve 7%).
 - ☒ New Active profile created. Old one preserved as historical record.

TC-MER-12: Search & filter merchants [ADMIN, MERCHANT_OPS]

1. Sidebar → Merchants. In the search box at the top, type Testing .
 - ☒ Only merchants whose legal name contains "Testing" remain visible.
2. Use the Status dropdown → pick ACTIVE . Risk dropdown → LOW .

- ☒ Combined filter narrows further.
3. Click **Clear filters** — all merchants reappear.

TC-MER-13: Create a store under a merchant (embedded form) [ADMIN, MERCHANT_OPS]

1. Open Testing Corporation → **Stores** tab → click + **Add Store**.
 2. Fill:
 - **Store Name (required):** Downtown Branch
 - **Region:** APAC
 - **Address:** 42 MG Road
 - **City:** Bengaluru
 - **State:** KA
 - **Pincode:** 560001
 - **Contact Person:** Ravi Kumar
 - **Contact Phone:** +91 9876543210
 3. Click **Create Store**.
- ☒ New store appears in the list.

TC-MER-13b: Same fields appear on the standalone Stores page [ADMIN, MERCHANT_OPS]

1. Sidebar → Stores → + **Add Store**.
- ☒ Form has the same 8 fields as TC-MER-13 plus a **Parent Merchant** dropdown at the top (since you're not already on a merchant page).

TC-MER-14: Update store status [ADMIN, MERCHANT_OPS]

1. From the store row, click **Deactivate** → confirm. Then **Reactivate**.
- ☒ Both transitions succeed. Status badge updates.

TC-MER-15: Sidebar hidden for non-MOPs roles

1. Log out. Log in as pops1 .
- ☒ Sidebar has **no** Merchants link and **no** Stores link.
2. Type <http://localhost:5173/merchants> in the URL bar.
- ☒ Forbidden page.

Module 3 — Terminal / POS Provisioning

What this module does: create the physical card-reader devices, attach them to a store, set their behaviour parameters, and monitor their health.

TC-TERM-01: Create a terminal in a store [ADMIN, POS_OPS]

Prereq: TC-MER-13 (a store exists).

1. Log in as pops1 . Sidebar → **Terminals** → + **New Terminal**.
2. Fill:
 - **Merchant:** pick Testing Corporation from the dropdown
 - **Store:** pick Downtown Branch (this dropdown only shows stores under the chosen merchant)
 - **TID (required, exactly 8 digits):** 12345678
 - **Brand/Model:** Ingenico iCT250

- **Capability:** EMV

3. Save.

- ✓ Terminal detail page (/terminals/<id>) opens. Top header shows 12345678 , status Active , the resolved store name and merchant name (not just numeric IDs).
- 🐞 SQL: SELECT tid, store_id, merchant_id FROM acquirerx_terminal.terminal WHERE tid='12345678'; — both store_id AND merchant_id are populated (not NULL).

TC-TERM-02: Duplicate TID rejected [ADMIN, POS_OPS]

1. Try to create another terminal with TID = 12345678 .

- ✓ Inline error in the modal: "Terminal with TID 12345678 already exists." (The error message is the **real** backend message, visible thanks to the error mapping fix.)

TC-TERM-03: TID format validation [ADMIN, POS_OPS]

1. Try TID = abc12 (5 chars, letters).

- ✓ Frontend rejects before submit: "TID must be exactly 8 digits".

TC-TERM-04: Edit terminal [ADMIN, POS_OPS]

1. From terminal detail → click **Edit** (top right).
2. Change Brand/Model → Verifone V200c . Capability → CTLS . Save.

- ✓ Header refreshes with new values.

TC-TERM-05: Deactivate / reactivate terminal [ADMIN, POS_OPS]

1. On terminal detail, click **Deactivate** in the status-actions bar → confirm.

- ✓ Status badge flips to Inactive . Action button changes to **Activate**.

2. Click **Activate**.

- ✓ Back to Active .

TC-TERM-06: Create a parameter profile (friendly form) [ADMIN, POS_OPS]

What's a "param profile"? A bundle of terminal configuration (chip kernel version, contactless tap-and-go limit, receipt copies, idle timeout, etc.). One profile can be assigned to many terminals.

1. Open any terminal detail → click the **Profile** tab.
2. Click + **New Profile** (top-right of the "Assign Different Profile" card).
3. The friendly form opens. Fill:

- **Profile Name (required):** STD_RETAIL_v1
- **EMV Kernel:** v1.4 (default — leave it)
- **Contactless Limit (₹):** 5000 (means tap-and-pay works without PIN below ₹5000)
- **Receipt Copies:** 2
- **Idle Timeout (sec):** 60
- **Auto-Batch Close (hr):** 23 (24-hr format — closes the batch at 11 PM)
- **Currency:** INR

4. Click **Create Profile**.

- ✓ Profile appears in the "Available profiles" dropdown. Behind the scenes the form values were serialized to JSON and saved.

TC-TERM-06b: Advanced raw-JSON mode [ADMIN, POS_OPS]

1. Same flow, but click the small **Advanced (raw JSON)** link at the top right of the form.
- ☒ Form fields collapse, replaced by a JSON textarea pre-filled with what you'd already entered.
2. Type bad JSON `{ "foo": }` and click Create.
- ☒ Inline error: "Parameters JSON is not valid JSON." Form stays open.
3. Fix to `{"foo": "bar", "customField": 42}` and Create.
- ☒ Profile saved with the custom payload.

TC-TERM-07: Assign param profile to terminal [ADMIN, POS_OPS]

1. On the terminal's Profile tab, in the dropdown labelled "Available profiles", pick `STD_RETAIL_v1 (v1)`.
2. Click **Assign**.
- ☒ The "Current Assignment" card at the top shows `STD_RETAIL_v1`.

TC-TERM-08: Health ping [ADMIN, POS_OPS]

1. Terminal detail → **Health** tab → click **Record Ping**.
2. Battery: `80`, Signal Strength: `95`. Save.
- ☒ "Last Seen" updates to now. The history table grows by one row.

TC-TERM-09: Sidebar hidden for non-POS roles

1. Log in as `mops1` → no Terminals link in the sidebar.
2. Direct URL `/terminals` → Forbidden page.

Module 4+5 — POS Switch + Fee Engine (Transactions)

What this module does: the actual card swipe. The Switch is the routing/auth layer (decides Approve/Decline). The Fee Engine computes scheme + interchange + acquirer fees on each approved txn. Together they're the heart of the system.

TC-TXN-01: Open a batch on a terminal [ADMIN, POS_OPS]

Why? Transactions can only be authorized on a terminal that has an OPEN batch. End-of-day, the batch closes and money flows.

1. Open terminal `12345678` detail. Scroll to the **Batch Control** card at the bottom.
2. Click **Open Batch**.
- ☒ Card shows a green "Open" status with the open time.

TC-TXN-02: Authorize a SALE (the happy path) [ADMIN, MERCHANT_OPS]

1. Sidebar → **Transactions** → **+ New Transaction** (top-right blue button).
2. Form fields:
 - **Terminal:** pick `12345678 – Verifone V200c – Downtown Branch` from the dropdown (only terminals with `ACTIVE` status appear — make sure you've completed TC-TXN-01 first so it has an open batch)
 - **Amount (₹):** `1500.00`
 - **Currency:** `INR` (read-only)
 - **PAN (Masked):** `453201*****0366` (this exact format)
 - **Transaction Type:** `SALE`
3. Click **Authorize Transaction**.

-  The page changes to a green **Result** screen with `APPROVED` , an Auth Code (e.g. `045123`), Response Code `00` , your terminal info, merchant info, network (e.g. `LocalSim`), and a risk score.
-  The risk score (typically 0–30 for normal txns) is computed from the active risk rules (Module 9).

TC-TXN-03: PAN format validation — strict regex `[ADMIN, MERCHANT_OPS]`

1. New Transaction → PAN: `234932433243*X` (12 digits then mask at end).
-  Inline error under the PAN field: "Format: 6 digits + 3–9 of * or X + 4 digits (e.g. 453201*****0366)". Form does NOT submit. **No backend call is made.**

TC-TXN-04: Authorize without an open batch `[ADMIN, MERCHANT_OPS]`

1. First close the batch (TC-TXN-09).
 2. Try TC-TXN-02 again on that terminal.
-  Yellow warning alert in the form: "No open batch on this terminal. Open a batch first from the terminal detail page."

TC-TXN-05: High-amount decline `[ADMIN, MERCHANT_OPS]`

1. New Transaction with amount `75000` (above the configured 50k risk threshold from the form hint).
-  Result page shows **DECLINED** with a response code, plus a Risk Reason like "AMOUNT_TOO_HIGH".

TC-TXN-06: Filter transactions list `[ADMIN, MERCHANT_OPS, RISK]`

1. Sidebar → Transactions. Scroll up — there are filters: Merchant, Store, Terminal, Date Range, Search (Auth/PAN), Status, Type.
 2. Set:
 - **Merchant:** `Testing Corporation Pvt Ltd`
 - **Store:** `Downtown Branch`
 - **Terminal:** dropdown → pick `12345678` (this dropdown should now actually populate; if it shows "No results" check that the terminal really belongs to that store)
 - **Status:** `Approved`
 - **Type:** `Sale`
 - **Date Range:** today's date for both From and To
-  The table filters down. Count and Total Amount cards update to match.

TC-TXN-07: Date range no longer crashes the page `[ADMIN, MERCHANT_OPS, RISK]`

1. With filters open, pick a From date and a To date by clicking inside the date input.
-  No "Failed to load transactions / Internal server error" red box. Same on Settlements / Disputes / Recon / Risk / Notifications pages — date pickers all work.

TC-TXN-08: Transaction detail `[ADMIN, MERCHANT_OPS, RISK]`

1. Click any approved row in the table.
-  Detail page shows: amount, PAN, merchant, terminal, network, **fee breakdown** (Scheme Fee, Interchange Fee, Acquirer Markup, Net Merchant Amount = amount – all fees).

TC-TXN-09: Close a batch `[ADMIN, POS_OPS]`

1. Terminal detail → Batch Control → **Close Batch**.
-  Batch row status flips to `CLOSED` . Any new auth on this terminal returns the "no open batch" error until a new batch is opened.

TC-TXN-10: Fee Rules CRUD [ADMIN]

What this is: the rules that decide what scheme/interchange/markup % to apply on each transaction. Matched by card type, txn type, network, region, MCC pattern, and amount range.

1. Sidebar → **Fee Rules** (under "Administration"). Click **+ New Rule**.
2. The modal opens (this modal is now scrollable; if it's a small screen the body scrolls and Save/Cancel stay visible at the bottom).
3. Fill:
 - **Card Type (required):** CREDIT
 - **Transaction Type (required):** SALE
 - **Scheme %:** 0.10
 - **Interchange %:** 1.50
 - **Acquirer Markup %:** 0.40
 - **Network:** V (Visa)
 - **Region:** APAC
 - **MCC Pattern:** leave blank (or 5411 to scope to grocery only, or 54* for any code starting with 54)
 - **Min/Max Amount:** leave blank for any amount
 - **Priority:** 100 (lower number = higher priority — 1 wins ties)

4. Click **Create Rule**.

- ☒ New row appears in the rules list. Effective immediately on the next authorize call.

Module 6 — Settlement, Payouts & Adjustments

What this module does: the once-daily process that bundles a merchant's approved transactions into a "settlement batch", deducts fees, and creates a "payout" record for the bank to pay the merchant. Adjustments are manual corrections (e.g. for chargebacks).

TC-SET-01: Run settlement for a merchant [ADMIN, RECON]

Prereq: at least one APPROVED transaction exists for the merchant (TC-TXN-02).

1. Log in as admin1 or recon1 . Sidebar → **Settlements**.
 2. Click the yellow **Run Settlement** button at the top right.
 3. Modal opens. Pick **Testing Corporation Pvt Ltd** from the merchant dropdown.
- ☒ The big yellow **Run Settlement** button at the bottom of the modal becomes clickable (no greyed-out state).
4. Click **Run Settlement**.
- ☒ Modal closes. A new row appears in the "Batches" table: #1 , Period, Merchant, Txn count, Gross, Fees (₹0 in this simple case), Net, Status PAID , Posted today.

TC-SET-01b: "Nothing to settle" friendly message [ADMIN, RECON]

1. Immediately re-run TC-SET-01 (without doing a new transaction in between).
- ☒ Modal stays open with a blue info alert: "All transactions for *Testing Corporation Pvt Ltd* are already settled. Run a new transaction first to create another settlement batch." **No red toast and no scary 'Failed' message.**

TC-SET-02: View settlement detail + payouts [ADMIN, RECON]

1. Click a row in the Batches list (or the **Open** link on the right).
- ☒ Detail page shows: Batch ID, Period start/end, Gross, Fees, Net, Status. Below that: list of Payouts (Bank Account Ref, Amount, Status `POSTED`).

TC-SET-03: Filter settlements [ADMIN, RECON]

1. Settlement list → set Status = `PAID` , Min Net = `1000` , Date Range = last 7 days.
- ☒ Filtered. Date picker does NOT crash the page (this used to bug out previously).

TC-SET-04: New manual adjustment [ADMIN, RECON]

1. Settlements → + **New Adjustment** (top right).
2. Modal opens with these fields:
 - **Merchant**: pick `Testing Corporation Pvt Ltd`
 - **Adjustment Type** (dropdown): pick one
 - `Reversal — undo a settlement`
 - `Fee Correction — fix wrong MDR/charge`
 - `Missing Payout — add a missed transaction`
 - `Bank Return — failed transfer reverted`
 - `Manual Credit — discretionary credit`
 - `Manual Debit — discretionary debit`
 - **Amount (₹)**: type a positive number, e.g. `250`
 - **Direction** (toggle): pick + **Credit** (adds to merchant) or – **Debit** (deducts)
 - **Transaction ID** (optional): the `auth_id` of an approved txn, or blank for a batch-level adjustment
 - **Reason / Justification (required)**: free text — **at least 20 characters**. Live counter shows how many more you need. Example: `Chargeback for txn #12 — customer claimed unauthorized swipe`
3. Click **Create Adjustment**.
 - ☒ Modal closes. Adjustment appears in the merchant's "Recent Adjustments" panel. The signed amount in the DB matches your direction (+ for Credit, – for Debit).

TC-SET-05: Adjustment validation [ADMIN, RECON]

1. Open New Adjustment. Leave Reason empty → Create.
 - ☒ Inline error: "Reason must be at least 20 characters".
2. Type a 10-character reason.
 - ☒ Counter shows red `10 more characters needed` . Submit blocked.
3. Set Amount = `0` or negative.
 - ☒ Inline error: "Amount must be positive". (Sign is set by the Direction toggle, never by typing a negative number.)
4. Leave Adjustment Type unselected.
 - ☒ "Adjustment type is required."

TC-SET-06: Sidebar hidden

1. Log in as `disp1` → no Settlements link.

Module 7 — Disputes

What this module does: when a cardholder claims "I never bought this", a dispute case is opened. The merchant has a deadline to either accept the chargeback or fight it with evidence. The case progresses through 4 stages: **Retrieval** → **Chargeback** → **Representation** → **Arbitration**.

TC-DISP-01: Open a dispute (using Postman) [ADMIN, DISPUTES]

The UI assumes the dispute already exists (in real life, the network creates it). To create one for testing, use Postman — see Section K for the full setup.

Quick form:

- **Method:** POST
- **URL:** `http://localhost:9090/api/v1/disputes`
- **Headers:** `Content-Type: application/json` , `Authorization: Bearer <your-jwt>`
- **Body:**

```
{ "txnId": 12, "panMasked": "453201*****0366", "reasonCode": "FRAUD_4837" }
```

Replace `12` with a real `auth_id` from `acquirerx_transaction.auth_message` .

After this, refresh the Disputes page — your dispute appears.

TC-DISP-02: List + filter disputes [ADMIN, DISPUTES]

1. Log in as `disp1` . Sidebar → **Disputes**.
- ☒ Table shows all disputes with Case ID, Merchant, Txn ID, Reason Code, Stage, Amount, Status, Deadline, Opened.
2. Filters at top: Stage, Status, Reason Code search, Merchant, Date Range, "Deadline expired" checkbox.
 3. Try Stage = `RETRIEVAL` , Status = `OPEN` , Reason Code search = `FRAUD` .
- ☒ Filters narrow correctly. Date picker does NOT crash.

TC-DISP-03: Stages strip + advance stage [ADMIN, DISPUTES]

1. Click any dispute row to open the detail page.
- ☒ At the top of the page below the title there's a **stage strip**: `1. RETRIEVAL` → `2. CHARGEBACK` → `3. REPRESENTMENT` → `4. ARBITRATION` . The current stage is highlighted blue, prior stages also blue, future stages grey.
2. On the right side, click **Advance Stage** (only visible to ADMIN/DISPUTES, only when status is OPEN, only when not already at ARBITRATION).
- ☒ Stage strip moves one step right (e.g. `RETRIEVAL` → `CHARGEBACK`).
 - ☒ When you've advanced to ARBITRATION, the Advance button greys out.

TC-DISP-04: Link a document (URI, not a file upload) [ADMIN, DISPUTES]

Per spec, documents are URI references — not file uploads. You give a URL where the doc lives.

1. On the detail page, scroll to the **Documents** card.
2. In the "Link a Document" form:
 - **Doc Type:** pick `RECEIPT` from the dropdown (others: `INVOICE` , `DELIVERY_PROOF` , `COMMUNICATION`)
 - **Document URI:** `https://files.example.com/receipts/RX-12.pdf`
3. Click **Link**.

- ☒ Row appears with the type badge, the URI as a clickable link.

TC-DISP-05: Log a dispute action [ADMIN, DISPUTES]

1. Right-side panel → "Log an action" form.
2. Action: pick `SUBMIT_EVIDENCE` from the dropdown. Other options: `REQUEST_DOCS` , `ACCEPT` , `REJECT` , `WRITE_OFF` , `ESCALATE` .
3. Notes (optional): `Submitted invoice + delivery proof for txn #12` .
4. Click **Log Action**.

- ☒ Row appears in "Action History" with the action type, your `userId` (from JWT), notes, timestamp.

TC-DISP-06: All action types

Repeat TC-DISP-05 with each of `REQUEST_DOCS` , `ACCEPT` , `REJECT` , `WRITE_OFF` , `ESCALATE` . Each adds a new history row.

TC-DISP-07: Close a dispute [ADMIN, DISPUTES]

1. Click **Close Dispute** (right panel).
- ☒ Status flips to `CLOSED` . The Actions panel disappears (no more actions on closed disputes). Stage strip stays at whatever it was.

TC-DISP-08: Sidebar hidden

1. Log in as `risk1` → no Disputes link.

Module 8 — Reconciliation & Exceptions

What this module does: at end of day, AcquirerX has its own record of every transaction. The bank also sends a file with their record. We compare. Mismatches are "exceptions" that an analyst resolves.

TC-REC-01: Load a reconciliation file [ADMIN, RECON]

1. Log in as `recon1` (or admin). Sidebar → **Reconciliation**.
2. Click + **Load File**.
3. Modal:
 - **Source:** pick `BANK` (other options: `SWITCH` , `NETWORK`)
 - **File Date:** today's date
 - **Items:** in real use you'd upload a CSV. For UI testing, paste this minimal JSON-style data:

```
[
  { "reference": "AUTH-101", "amount": 1500.00, "externalRef": "NEFT-9981" },
  { "reference": "AUTH-102", "amount": 2500.00, "externalRef": "NEFT-9982" }
]
```

4. Submit.

- ☒ A new "Recon File" row appears in the Files tab with status `PROCESSED` , RowCount `2` .

TC-REC-02: View match status of file items [ADMIN, RECON]

1. Click the file row to open it.
- ☒ List of items each with `MatchStatus` = `MATCHED` / `UNMATCHED` / `MISMATCHED` . (Match logic compares the bank's reference + amount against AcquirerX's records.)

TC-REC-03: Filter file list by source / date [ADMIN, RECON]

1. Recon page → Files tab → set Source = SWITCH , date range = last 7 days.
- ☒ Filters apply. **No page crash on date pick** (this used to bug out previously).

TC-REC-04: Resolve an exception (Mark Resolved or Write Off) [ADMIN, RECON]

Where the workflow lives: the Reconciliation page has THREE tabs at the top: Files Items Exceptions . The yellow badge on **Exceptions** shows the count of unresolved cases.

1. Sidebar → Reconciliation. Click the **Exceptions** tab.
- ☒ Table appears with columns: Exception ID , Reference , Category (MissingTxn / Duplicate / AmountMismatch), Created , Status , and a Resolve button on each OPEN row.
2. **Mark Resolved flow:**
 1. Click **Resolve** on any OPEN row. A modal opens.
 2. Pick the green **Mark Resolved** option ("Item is reconciled — any required follow-up has been completed").
 3. Type notes — **at least 15 characters**. Example: Bank credit confirmed in next-day statement.
 4. Click **Confirm**.
 - ☒ Modal closes. Row's status flips to RESOLVED . Match-Rate donut and counters refresh.
3. **Write Off flow:**
 1. Click **Resolve** on a different OPEN row.
 2. Pick the yellow **Write Off** option ("Discrepancy cannot be reconciled and is being absorbed").
 3. Type notes — **at least 30 characters**. Example: Bank confirmed reversal not possible — absorbed by acquirer per policy 3.2.
 4. Click **Confirm**.
 - ☒ Status flips to WRITTEN_OFF .
4. Use the Status filter at the top of the Exceptions tab → pick Resolved to confirm rows with status RESOLVED appear; same for Written Off .

***i** The Resolve button only appears for ADMIN/RECON. Other roles don't see the column at all — that's the RBAC guard, not a missing feature.*

TC-REC-05: Bad source rejected (Postman test, optional)

1. POST /api/v1/recon/load with source=GOOGLE .
- ☒ "Source must be SWITCH, NETWORK, or BANK".
-

Module 9 — Risk & Fraud

What this module does: rules and blacklists that decide whether to allow, review, or block a transaction at authorization time. An analyst can review fired risk events and tune the rules.

TC-RISK-01: Create a risk rule [ADMIN, RISK]

1. Log in as risk1 . Sidebar → Risk → Rules tab → + New Rule.
2. Fill:
 - **Name:** High amount review
 - **Expression** (optional, free text): amount > 50000

- **Max Amount:** 50000
- **Severity:** pick HIGH
- **Action:** pick REVIEW

3. Save.

- ✓ Rule appears in the list with Active badge.

TC-RISK-02: Rule severity validation [ADMIN, RISK]

1. Create new rule, Severity = URGENT .
- ✓ Error: "Severity must be LOW, MEDIUM, HIGH, or CRITICAL".

TC-RISK-03: Deactivate a rule [ADMIN, RISK]

1. Click the **Deactivate** button on a rule.
- ✓ Status flips to INACTIVE . Future authorizations stop matching it.

TC-RISK-04: Rule fires on a transaction [ADMIN, RISK]

1. Reactivate the rule from TC-RISK-01.
 2. Log in as mops1 (in another tab). Run a SALE transaction with amount 60000 (above the 50k threshold).
 3. Switch back to the risk1 tab → Risk → **Events** tab.
- ✓ A new event row: rule name "High amount review", the txn id, decision REVIEW or BLOCK , score, event date.

TC-RISK-05: Add a PAN to the blacklist [ADMIN, RISK]

1. Risk → **Blacklist** tab → + **New Entry**.
 2. Type = PAN , Value = 453201*****9999 , Reason = Confirmed fraud chargeback on 2026-04-15 .
 3. Save.
- ✓ Row added.

TC-RISK-06: Blacklisted PAN is blocked at auth [ADMIN, RISK]

1. As mops1 , try to authorize a SALE with PAN = 453201*****9999 .
- ✓ Result page shows DECLINED . Risk Reason mentions blacklist.

TC-RISK-07: Filter risk events [ADMIN, RISK]

1. On Risk → Events tab, set date range = last 7 days, Decision = BLOCK .
- ✓ Filters work. No page crash on date pick (previously it would crash here).

TC-RISK-08: Sidebar hidden

1. Log in as recon1 → no Risk link in the sidebar.

Module 10 — Reports & Dashboards

What this module does: aggregated views of the system. The Dashboard is the tile-based home page; the Reports page lists generated AcquirerReport records.

TC-RPT-01: Dashboard counters [any role]

1. Log in as any role. The Dashboard is the default landing page.

- ☒ Tiles populate (txn count, approval rate, fee yield, chargeback rate). For MERCHANT_OPS the tiles are scoped to their merchants only.

TC-RPT-02: Reports list [ADMIN, MERCHANT_OPS, RECON]

1. Sidebar → **Reports**.
- ☒ Existing AcquirerReport rows shown with Scope (Merchant/Network/Period), Metrics (Volume/Value/Fees/Net/ChargebackRate), Generated date.

TC-RPT-03: Generate a merchant report [ADMIN, MERCHANT_OPS]

1. Reports → + **New Report**.
 2. Scope: Merchant . Pick Testing Corporation Pvt Ltd . Period: last 7 days. Generate.
- ☒ New report row appears with the metrics filled.

TC-RPT-04: Network-scope report [ADMIN]

1. Same as TC-RPT-03 but scope = Network .
- ☒ Aggregates by V/M/U/LocalSim.

TC-RPT-05: Sidebar hidden

1. Log in as disp1 → no Reports link.

Module 11 — Notifications

What this module does: in-app notifications (the bell icon). System-generated events: settlement posted, dispute deadline approaching, risk block fired, recon failure, batch closed.

TC-NOTIF-01: Bell icon unread count [any role]

1. Trigger an event (e.g. recon1 runs a settlement (TC-SET-01), or disp1 opens a dispute).
2. Within ~30 seconds, the bell icon at the top right shows a red badge with a count.

TC-NOTIF-02: Notifications page [any role]

1. Click the bell icon, or sidebar → **Notifications**.
- ☒ List shows newest first. Each row has a Category badge (Batch / Settlement / Dispute / Risk / Recon), a Status (Unread / Read / Dismissed), and a timestamp.

TC-NOTIF-03: Mark a notification as read [any role]

1. On a notification row, click **Mark Read**.
- ☒ Bell badge count drops by 1. Row's status flips to Read (or it disappears from the unread filter view).

TC-NOTIF-04: Dismiss a notification [any role]

1. Click **Dismiss**.
- ☒ Notification disappears from the active list.

TC-NOTIF-05: Mark all as read [any role]

1. Click **Mark all read** (top of the list).
- ☒ Bell badge clears.

TC-NOTIF-06: Filter by category and date [any role]

1. Set filter Category = `DISPUTE` , date range = last 7 days.

-  Filters apply. Date picker does NOT crash (was crashing previously).

Section I — End-to-end happy-path walkthrough

Run this once you've smoke-tested every module. It's a single 16-step scenario that exercises the full system: a merchant gets onboarded, a sale happens, a high-value sale gets risk-blocked, a settlement runs, a dispute is opened and walked, a report is pulled. If every step succeeds with no red toast or error screen, the system is broadly healthy.

#	As role	Action	Expected
1	n/a	Register all 6 test users (Section G)	All 6 can log in
2	ADMIN	Onboard merchant (TC-MER-01)	Status Pending
3	ADMIN	Submit KYC (TC-MER-03)	Merchant flips to Active
4	ADMIN	Add pricing model (TC-MER-07)	Active pricing row
5	ADMIN	Add settlement profile (TC-MER-10)	Active profile row
6	MERCHANT_OPS	Create store (TC-MER-13)	Store visible
7	POS_OPS	Create terminal in store (TC-TERM-01)	DB row has both <code>store_id</code> and <code>merchant_id</code>
8	POS_OPS	Open batch on terminal (TC-TXN-01)	Batch status Open
9	MERCHANT_OPS	Authorize SALE ₹1500 valid PAN (TC-TXN-02)	APPROVED
10	MERCHANT_OPS	Authorize SALE ₹75000 (TC-TXN-05)	DECLINED + Risk Event logged
11	RISK	See the new Risk Event in Events tab (TC-RISK-04)	Event row exists
12	POS_OPS	Close batch (TC-TXN-09)	Batch closed
13	RECON	Run settlement (TC-SET-01)	Batch PAID with the ₹1500 net
14	RECON	Load bank file with the netted amount (TC-REC-01)	File Processed, items Matched
15	DISPUTES	Open dispute via Postman, walk through stages, close (TC-DISP-03 → 07)	Dispute closed at some stage
16	MERCHANT_OPS	Generate merchant report for today (TC-RPT-03)	Report row with the day's metrics

Pass criteria for the whole walkthrough: no toast says "Something went wrong on our side", no `ErrorBoundary` screen appears, every step's expected outcome is observed.

Section J — Negative & security tests (RBAC)

These are the tests that prove the system is hardened. Run them as a habit after every code change.

#	Step	Expected
J-1	Log in as RISK, paste <code>/admin/users</code> in URL	Forbidden / redirected
J-2	Log in as DISPUTES, paste <code>/settlement</code>	Forbidden
J-3	Log in as POS_OPS, paste <code>/merchants/new</code>	Forbidden
J-4	Log in as MERCHANT_OPS, paste <code>/risk</code>	Forbidden
J-5	Log out, paste <code>/dashboard</code>	Redirected to <code>/login</code> , with the original URL stored to come back to
J-6	F12 → delete <code>ax_token</code> from Local Storage → click anywhere	Auto-redirected to <code>/login</code>
J-7	F12 → set <code>ax_token</code> to a garbage string → click anywhere	401 toast → redirected to <code>/login</code>
J-8	Register username with spaces (<code>kail kail</code>)	"Username can only contain letters, numbers, and underscores"
J-9	Register email = <code>not-an-email</code>	"Email should be valid"
J-10	Submit a transaction with raw PAN <code>4532012345670366</code> (16 digits, no mask)	Frontend zod blocks: "Format: 6 digits + 3–9 of * or X + 4 digits..." — no backend call
J-11	Try to deactivate yourself (User Mgmt → your row → Deactivate)	"You cannot deactivate your own account"
J-12	Try to change your own role	"You cannot change your own role"
J-13	Open ProfileTab → empty Profile Name → Create Profile	Inline validation prevents submit
J-14	Adjustment Reason = 5 chars (TC-SET-05)	"Reason must be at least 20 characters"

Section K — How to use Postman with this system

A few tests in the playbook (TC-DISP-01, TC-REC-05) can't be done from the UI — they need direct API calls.

Step 1 — Login to get a JWT token

Field	Value
Method	POST
URL	<code>http://localhost:9090/api/v1/auth/login</code>
Headers	Content-Type: <code>application/json</code>
Body (raw JSON)	<code>{ "username": "admin1", "password": "Admin@123" }</code>

The response will look like:

```
{
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiJ9...long.string",
    "username": "admin1",
    "role": "ADMIN",
    "message": "Login successful"
  }
}
```

Copy the `token` value (the long string).

Step 2 — Use the token on every other request

For every other API call, add this header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...long.string
```

Time-saver — auto-save the token in Postman

1. On the Login request, go to the **Tests** tab and paste:

```
const data = pm.response.json().data;
pm.collectionVariables.set("token", data.token);
```

2. On every other request, set the Authorization header to: `Bearer {{token}}` . Postman auto-substitutes the saved token.

Example: open a dispute via Postman

Field	Value
Method	POST
URL	http://localhost:9090/api/v1/disputes
Headers	Content-Type: application/json Authorization: Bearer {{token}}
Body (raw JSON)	{ "txnId": 12, "panMasked": "453201*****0366", "reasonCode": "FRAUD_4837" }

Replace `12` with a real `auth_id` — get one with: `SELECT auth_id FROM acquirerx_transaction.auth_message WHERE status='APPROVED' LIMIT 5; .`

Common Postman errors and what they mean

Status	What it means	What to do
401 Unauthorized	Token missing, expired, or wrong	Re-login (Step 1), copy the new token

403 Forbidden	You're authenticated but your role isn't allowed	Log in as ADMIN
400 Bad Request, field-level error	Validation failed (PAN format, MCC format, etc.)	Read the <code>fieldErrors</code> in the response body, fix the input
404 Not Found	The resource (txn, merchant, dispute) ID doesn't exist	Pick a real ID from the DB
503 Service Unavailable	The microservice that handles this URL hasn't registered with Eureka yet	Wait 30 seconds, retry. Check http://localhost:8761 to see which services are missing

Section L — Troubleshooting

"Something went wrong on our side. Please try again." red toast

- A backend service crashed or is throwing an unhandled exception. Check the **terminal logs** of the relevant microservice (auth-service, merchant-service, etc.).
- If you see a Java stack trace, copy the first line and the "Caused by" line into your bug report.

Browser stuck on a white screen / "Something went wrong"

- That's React's `ErrorBoundary` catching a render-time crash. Open F12 → Console tab → look for the red error.
- Hard refresh (`Ctrl + Shift + R`) often clears it.

Eureka shows a service as missing

1. Is the service's terminal actually running? Look for `Started X in N seconds`.
2. Did MySQL start before the service? Auth-service crashes on boot if it can't reach the DB.
3. Restart that one service — `Ctrl+C` in its terminal, then `mvn spring-boot:run` again.

Frontend can't log in / 401 on every request

- Check <http://localhost:8761> — auth-service must show `UP`.
- Check the browser DevTools → Network tab → find the `/auth/login` request → response code. If it's CORS-related, the api-gateway isn't running.
- Clear local storage: F12 → Application → Local Storage → right-click → Clear.

Date picker crashes the page (ErrorBoundary)

- This was a known bug, fixed. If you see it again, note which page and which date input, file a bug.

"Cannot sort by 'X'" toast

- A frontend pagination call is sending an invalid `sortBy`. The toast lists allowed fields. File a bug, mention the page you're on.

Hibernate "Unknown column" exceptions in service logs

- The DB schema is out of sync. Run that microservice; on boot, with `ddl-auto=update`, Hibernate adds missing columns.
- If it doesn't fix automatically, drop the database and let it recreate: `DROP DATABASE acquirerx_X; CREATE DATABASE acquirerx_X;` then restart that service.

"Account is inactive" — how do I unblock myself?

- An admin must reactivate you (TC-IAM-09). If your only admin account is also inactive, you need to update the DB directly:

```
UPDATE acquirerrx_auth.app_user SET status = 'ACTIVE' WHERE username = 'admin1';
```

Token expired

- Default JWT expiry is 24 hours. Just log out and log in again to get a fresh token.

Merchant settlement says "Failed to run settlement"

- 99% of the time the message says "No unsettled transactions found" — you've already settled everything. Run a new transaction first.
- After our recent fix, this should show as a friendly blue info alert, not a red error. If you see a red error here, file a bug.

Terminal dropdown shows "No results"

- The terminal must belong to the store you've selected. If your test terminals are in store #7 but you've filtered by store #5, there's nothing to show — that's correct behaviour.
- If you're certain the terminal exists in the selected store but it's not showing, file a bug.

"Cannot deactivate your own account"

- Working as designed. Use a second admin account to deactivate yourself if you really need to.

Section M — Bug-report template

When you find an issue, copy-paste this template into Slack/Jira/wherever:

```
**Module:** <e.g. Module 4 – Transactions>
**Test case ID:** <e.g. TC-TXN-04>
**Role logged in as:** <ADMIN | MERCHANT_OPS | POS_OPS | RISK | DISPUTES | RECON>
**Browser + version:** <e.g. Chrome 124>

**Steps to reproduce:**
1. ...
2. ...
3. ...

**Expected:**
<what the playbook says should happen>

**Actual:**
<what really happened>

**Console errors (F12 → Console):**
<paste any red errors – copy the full message>

**Network response (F12 → Network → click the failed call):**
- Status code: <e.g. 500>
- Response body: <paste the JSON>
```

```
**Screenshot:** <attach>
```

```
**Service logs (if backend issue):**
```

```
<paste the relevant lines from the microservice terminal>
```

The richer the report, the faster the bug can be reproduced and fixed without back-and-forth.

Final word

If anything in this playbook doesn't match what you actually see in the UI, **the playbook is wrong, the UI is right** — please open a bug against the playbook so it gets corrected. The code evolves faster than docs, so spotting drift is helpful.

Good testing. 🐛 ✨